

Smith/FCAD Python Cheat Sheet

James Lowenthal/Mike Petersen

Python:

Python is a freely-available, dynamic, high-level programming language designed to be easily understandable. It is rapidly becoming a standard programming language in research astronomy. This Cheat Sheet assumes Python version 3.

Starting Python from a Smith College Macintosh computer with Anaconda installed

Log into a machine using your Smith Novell account. Ensure that Terminal is running (found on the Dock or in the Utilities folder). In Terminal, type `python`. Now your Terminal should have a `>>>` prompt. You can test it by typing `1+1` at the prompt to do a simple arithmetic calculation. To exit the python interpreter, type `>>> exit()`.

However, to run software you write yourself, you will most often be executing python scripts (files ending in `.py`). These are executed from the Terminal window (not the Python interpreter!) by typing `> python my_script.py`

Organization: Python is composed of...

```
...variables: >>> mydata = [ 4, 5, 1, 2, 3, 4 ]
...libraries of functions: >>> import numpy
...functions: >>> myanswer = numpy.mean( mydata )
               >>> print ( ' The mean of mydata is: ' , myanswer)
...or for Python v2: >>> print ' The mean of mydata is: ' , myanswer
```

Comments

Comments are lines (or parts of lines) of code that are not read by the code interpreter. They are meant for humans to read, usually to explain what the code is doing. In Python, single-line comments begin with `#`, while comments that are multiple lines use `'''` to begin and end:

```
>>> print ( ' Print this ' ) # but don't print this
```

Script Structure: Indentation and Line Continuation

Python relies on strict structure for many aspects of interpretation. After every colon used to control flow, the next line must be indented. These indents stack; for example, if an indented line ends with a colon, the next line must have two indents. Indents are added by using the tab key. A line can be continued (that is, read as one continuous line by the interpreter) using `\`. Examples are in the 'Conditional Statements' and 'Loop Statements' sections below.

Python Libraries

In the above organization, we used the numpy library. To import libraries, use `>>>import library_name`. You may also rename these libraries; a common shorthand for numpy is np, defined like this: `>>>import numpy as np`. Matplotlib is another common library, with an often-used shorthand `>>> import matplotlib.pyplot as plt`.

Getting Information about Python:

You can get information about different Python functions by typing `>>> help(function)` in the interpreter. However, it is often far easier to explore the wealth of online documentation, particularly when writing a script file.

If you want to see all the value(s) stored in a variable named mydata for example, you can print it to the terminal by entering: `>>> print (mydata)`. Also try just `>>> mydata`

Variables and Arrays

Python has a variety of data types: bytes, short integers, long integers, floating, double, complex, and strings. Variables should start with a character and may continue either with characters, numbers or '_'. Variables names cannot use existing Python commands (e.g. “if”, “else”, etc.).

Python variables and arrays are defined dynamically and you do not need to declare them at the beginning. For example, the *type of a variable or array may change* several times along an Python script. In principle, this is an advantage, but be careful since it is very easy to make mistakes! To see what type of variable Python thinks a particular variable is:

```
>>> type (mydata)
```

Conditional Statements (note indentations following colons)

```
myvalue = 10 # change to 0 and re-try!
```

```
if (myvalue > 1):
    print ('Greater than 1.')
else:
    print ('Less than or equal to 1.')
```

```
myvector = [0,5,2,5,78,8,3,45] # Creates variable of data type "list"
import numpy as np
myvector_array = np.array(myvector) # Change list type to array type
large = np.where(myvector_array >= 10)[0]
# Where searches array for positions of elements matching given
condition, i.e. value >= 10.
# Owing to the possible returns, [0] needed at the end of the where
call
print('large = ', large) # Print out the array positions of interest
```

```
print('large points to these values: ', myvector_array[large]) # print
the array values of interest
```

Loop Statements

```
# range is a function that sets up a list to iterate
for index in range(1,10):
    print ('This is iteration %i of 10 in your for loop!' %index )

index=long(0) # use a long integer to allow for large values!
while index <= 1000000:
    index *= 2 # Could also use index = index * 2
print (' Is index = %i less than or equal to 1,000,000?\n
    If so, your while loop continues!' %index )
```

Note that line continuations are not sensitive to spaces, but will not permit indentations.

Selected example tasks in Python

Read the first four columns of an ASCII (text) table into four Python arrays named column1, column2, column3, column4 with format types string (a), integer (i), float (f), and double (d) :

```
>>> import numpy as np
>>> Data = np.loadtxt( 'filename', dtype={'names': ('column1',
'column2', 'column3', 'column4'), 'formats': ('a', 'i', 'f4', 'f8')})
```

Note that after the format letter, the number specifies the number of bytes to allocate. This method creates a Python ‘dictionary’, which is accessed like so:

```
>>> print (Data['column1'])
```

Read a FITS file's primary image data and header data into two Python arrays, “image” and “header”:

```
>>> from astropy.io import fits
>>> image,header = fits.getdata(fitsfilename, header=True)
```

Element-by-element arithmetic is fast and easy in Python, if the arrays involved are the same size:

```
>>> mydiff = image - image2 ; Subtract one array from another
>>> mysum = image + image2 ; Add one array to another
>>> myproduct = image * image2 ; Multiply one array with another
>>> myratio = image / image2 ; Divide one array by another
```

Display an image in a matplotlib window:

```
>>> import matplotlib.pyplot as plt
>>> plt.ion() # this turns interactive plotting on
>>> plt.imshow( image - np.median(image) )
```

Plot a graph of the vectors xdata and ydata using asterisks:

```
>>> import matplotlib.pyplot as plt
>>> plt.plot( xdata, ydata, marker='o')
>>> plt.axis([xmin,xmax,ymin,ymax]) # don't forget to define the
variables!
>>> plt.xlabel('xdata')
>>> plt.ylabel('ydata')
>>> plt.title('plot title')
>>> plt.show() # Show plot on screen; close new plot window to regain
control of interpreter
```

Plot a histogram of the array data with 5 bins:

```
>>> import matplotlib.pyplot as plt
>>> plt.hist(data, 5)
```

Save as a .png :

```
>>> import matplotlib.pyplot as plt
>>> plt.plot( xdata, ydata, marker='o')
>>> plt.savefig('/path/to/figuresave.png')
```

Basic Program Construction

To make a script (a simple collection of Python commands), in your favorite text editor, type:

```
'''
plot_example - A script that demonstrates some plotting features.
A name and brief description of the program at the top of the file is
always helpful.
'''
import numpy as np
import matplotlib.pyplot as plt
xdata = np.arange(0.,10.,1.) ; ydata = xdata**2.
# The decimals tell python to do everything at floating-point
precision
# ** is the power operator. For arrays, use np.power()
# Note that you can put more than one valid Python command on one line
using a semicolon ( ; ).
plt.plot(xdata,ydata)# Construct the plot of xdata vs. ydata
plt.title('Y=X$^2$') # matplotlib understands LaTeX!
#Caution: use fresh-typed single quotes after cut-and-paste of above
line
plt.show() # Show plot on screen; close new plot window to regain
control of interpreter
```

... and save the document as anything you like, (I use `plot_example.py` here). Then you can run this file by simply entering: `> python /where/i/put/plot_example.py`

To make a minimal Python function, in your text editor, type:

```
# hello_world - The first program anyone has to write when learning a
new programming language!
def hello_world(): # no inputs
    print (' Hello World! ') # don't forget the indent!
```

... and save the document as `hello_world.py` . In the same UNIX directory, you can access this definition by using

```
>>> from hello_world import hello_world
>>> hello_world()
```

Another example:

```
# roll_dice - Return the results from ndice simulated six-sided dice
rolls. Use ndice = 5 for Yahtzee!
import numpy as np
def roll_dice(ndice):
    # numpy.random.uniform() returns random numbers between 0 and 1
    random_num = int(np.random.uniform() * 6. + 1. )
    print (random_num)
    # You may return the variable using 'return'
    return random_num
```

Definitions can be included in a script file at the beginning, and will then be accessible elsewhere in the script simply by typing, e.g.,

```
roll_dice(5)
```